

VT Bot Battle Arena

Workshop Specification v1.0

Protocol vt-arena/1 • Game rps-v1 • September 16, 2026

Purpose: A collaborative 60–90-minute agentic-coding exercise for 10–15 Virginia Tech Computer Science graduate students. One team builds the arena; three teams build independently submitted containerized bots.

Scope: The original repeated rock-paper-scissors concept, presented as a robot tournament. This version has no physics, movement, weapons, damage, network services inside bots, or live LLM calls during matches. Coding assistants help participants create the software. The bots themselves are ordinary programs.

Status: This is a proposed implementation contract, not an existing arena product or a security certification. The kit contains a reference bot and fixtures, not an implemented arena. Numerical limits below are workshop design choices. Docker and Python implementation facts are supported by the official references at the end.

1. Team ownership

Give the arena team 4–6 people and each bot team 2–3 people. At 10 participants this can be 4+2+2+2; at 15 it can be 6+3+3+3.

Team	Owns	Minimum deliverable
Arena	Container lifecycle, protocol validation, referee, tournament scoring, event log, presentation	Run all pairings, enforce the contract, print standings, save replayable events
Bot A	Its own source, Dockerfile, manifest, tests	A qualified image using one strategy
Bot B	Its own source, Dockerfile, manifest, tests	A qualified image using another strategy
Bot C	Its own source, Dockerfile, manifest, tests	A qualified image using another strategy

Suggested strategies, not restrictions: recent-frequency prediction; repeated-sequence or transition prediction; an adaptive combination of simple predictors. A seeded random bot and an always-rock or cycling bot are useful unscored practice opponents. Only the three student bots enter the scored tournament.

Freeze this contract before coding. Bot teams must be able to work from fixtures while the arena is under development. No team may change shared protocol fields unilaterally.

2. Battle rules

- A **round** is one simultaneous move by each of two bots.
- A **match** consists of 100 rounds, unless ended by a forfeit or infrastructure cancellation.
- A **tournament** is a double round robin: A–B, A–C, B–C, then the same pairings with display slots reversed. Six matches total, 600 scheduled rounds; each bot plays four matches and up to 400 rounds.
- Legal moves are exactly the lowercase strings `rock`, `paper`, and `scissors`.
- Rock beats scissors, scissors beats paper, and paper beats rock. Equal moves tie.
- A round win adds one to the winner's round score; the loser gets zero. A tied round adds no points to either score, but increments the tie count.
- The bot with more round wins after round 100 wins the match. Equal round scores produce a drawn match.
- Match league points: win = 3, draw = 1 each, loss = 0. A forfeit win is worth 3. A double forfeit awards 0 to both.
- Rank by total league points, then league points in matches among the tied bots, then fewer forfeited matches. If still equal, share the place. Round margin and execution speed are displayed only, not ranking tie-breakers.
- Reset each bot with a fresh container for every match. No persistent state between matches. In-memory learning within a match is allowed.
- Use an independently generated private seed for each bot in each match. Do not give bots the opponent's seed or a common tournament seed. Record seeds in an organizer-only log and release them after the tournament if desired.
- A bot must use its supplied seed for all strategy randomness and not depend on wall-clock time, external files, hardware randomness, or prior matches. Matching seeds and inputs support repeatable bot behavior, but do not guarantee identical host timing.

The bot may use only the current round number, total round count, its own seed and state, the score, and completed rounds supplied by the referee. There is no live access to the opponent's current move. No human intervention or image updates during a scored tournament.

Simultaneous commitment

Dispatch each bot's request concurrently, starting that bot's deadline immediately before the write/flush. Collect both responses or their faults. Do not resolve, publish, or send feedback about either move until both response outcomes are known. The first valid response is irrevocable; extra or unsolicited stdout messages are protocol faults. The next request contains only already-resolved history. The referee is trusted, so a cryptographic commit/reveal protocol is not needed for this workshop.

Randomness lesson

Against an opponent choosing independently and uniformly each round, any move has a one-third chance to win, a one-third chance to tie, and a one-third chance to lose. Historical pattern detection cannot predict an independent next choice. Tournament results are classroom game outcomes, not evidence that one coding assistant is generally better than another.

3. What the audience sees

The minimum live view is a terminal scoreboard. A simple browser replay is the first presentation enhancement, not a dependency of the referee.

Display two bot names/avatars, round number, the last pair of committed moves, the round winner, cumulative round scores, ties, and clear timeout/forfeit notices. Below this, show tournament standings and remaining pairings. Moves stay hidden until both commits are complete. Any robot punch, shield, or hit animation is cosmetic and must not change scoring.

An illustrative screen:

VT BOT BATTLE ARENA		Match 2 of 6
Pattern Hawk	vs	Counter Gobbler
Round 30 / 100		
ROCK		SCISSORS
Round winner: Pattern Hawk		
Round wins: 13		Round wins: 11
Tied rounds: 6		
LEAGUE: Bot Matches W-D-L Points Forfeits		

Use a host-generated event log as the source of truth. Run matches without animation delays; play the recorded events at a readable speed afterward. A replay must not launch bots or affect a score. A browser view should escape bot names and log content rather than rendering arbitrary submitted HTML.

4. Architecture and transport

Run the arena as a trusted Python host process on one designated computer. Only the arena has Docker access. At a time, it runs the two bot containers needed for one match. The third bot is registered but idle. Keeping the referee outside a container avoids needing to mount the Docker socket into it.

Use bidirectional **UTF-8 newline-delimited JSON over stdin/stdout**, not bot HTTP endpoints. The arena writes requests to the container's stdin. The bot emits responses on stdout. Stderr is diagnostic output, kept separate. Containers do not talk to one another and have no external network access.

Start each bot once per match and reuse its process for all rounds. Do not create a container or call `docker exec` for every move. Use interactive stdin without a pseudo-terminal (`-i`, not `-t`). Docker supports attached standard streams; `--network=none` leaves only a loopback interface. [1,2]

5. Protocol: vt-arena/1

Display note: JSON examples may be indented for readability. Transmit each message as one compact JSON object on one line.

All messages are one compact JSON object per line, followed by a newline. Multi-line JSON, blank output lines, stdout banners, duplicate keys, NaN/Infinity, and unsolicited messages are invalid. Flush responses immediately. Fields are case-sensitive. Unknown fields are rejected by the arena for v1; an extension requires a protocol revision. Schemas are in schemas/.

Limits: each bot response is at most 4,096 UTF-8 bytes including the newline; each arena request is at most 65,536 bytes including the newline. All round numbers are one-based integers. JSON schemas describe shapes; the arena must also validate order, matching identifiers, score/history consistency, legal outcomes, counts, and deadlines.

5.1 Start request and readiness

Arena to bot:

```
{
  "type":"start", "protocol":"vt-arena/1", "game":"rps-v1",
  "match_id":"m01", "bot_id":"pattern-hawk", "seed":104729,
  "rounds":100, "move_timeout_ms":1000
}
```

Bot to arena:

```
{"type":"ready","protocol":"vt-arena/1","match_id":"m01"}
```

seed is an unsigned 32-bit integer. rounds is 100 for scored matches; qualification fixtures may use 1–100. A bot must initialize only from this request and its image, then acknowledge readiness. The arena has a 10-second startup/readiness budget per bot, beginning when it launches that bot with the image already local. No image pulls or builds take place within this budget.

5.2 Turn request and move response

The following is a complete hypothetical request for round 3. In a real match, all historical entries must be exactly the already committed moves.

```
{
  "type":"turn", "match_id":"m01", "round":3,
  "score":{"you":1,"opponent":1,"ties":0},
  "history":[
    {"round":1,"you":"rock","opponent":"scissors","result":"win"},
    {"round":2,"you":"paper","opponent":"scissors","result":"loss"}
  ]
}
```

Shown indented for readability only; transmit it on one line. Round 1 has an empty history and zero scores. On round r , history contains exactly $r-1$ entries in order. Its score counts are from the receiving bot's perspective. result is win, loss, or tie for that bot. The opponent receives a mirrored perspective.

Bot response:

```
{"type":"move","match_id":"m01","round":3,"move":"paper"}
```

Exactly one move response is allowed for the outstanding turn. Echo the request's match ID and round. The arena, never the bot, determines the winner and score. There is no separate per-round result message: feedback arrives through the next turn's full history.

5.3 End request

```
{
  "type": "end", "match_id": "m01", "reason": "completed", "outcome": "win",
  "score": {"you": 41, "opponent": 35, "ties": 24},
  "last_round": {
    "round": 100, "you": "paper", "opponent": "rock", "result": "win"
  }
}
```

reason is completed, forfeit, double_forfeit, or cancelled. outcome is the receiver's win, loss, draw, or no_contest. For a double forfeit both receive loss; for cancellation both receive no_contest. last_round is the last fully resolved round, or null if none. This supplies final-round feedback without an extra response cycle. The arena sends end best-effort to surviving processes, closes stdin, allows 2 seconds for shutdown, and forcibly removes any remainder. No stdout response is allowed. Cleanup after end does not change a finalized result.

6. Errors, deadlines, and bounds

Condition	Arena action
One bot exceeds readiness or its 1,000 ms move deadline	That bot forfeits the match
Invalid JSON, illegal move, mismatched ID/round, unexpected stdout, wrong protocol, or oversized response	Offending bot forfeits the match
Bot exits unexpectedly, exceeds memory/process bounds, or fails to start due to its image	Bot forfeits; qualification should catch most such failures
Both bots fault in the same unresolved round or startup phase	Double forfeit; 0 league points each
Docker daemon failure, host failure, or referee bug	No contest; fix the infrastructure and replay the whole match with the same images and seeds
Bot will not exit after an end message	Force-remove it; keep the finalized result

Preserve completed round history when a match is forfeited. Do not invent moves or award fictitious remaining-round scores. A double forfeit cannot be converted to an ordinary draw. Await the other bot's response/deadline before classifying a same-round single versus double fault.

Enforce limits on raw bytes while reading, not only after an entire JSON message has arrived. Cap total stdout at 512 KiB per bot per match and total stderr at 64 KiB; exceeding either is a bot fault. Drain both streams concurrently; keep buffers/queues bounded. Do not simply block on `readline()` or wait for process exit while ignoring another pipe. Python documents potential deadlocks when subprocess pipes fill. For a persistent protocol, use concurrent readers with explicit timeouts; do not call `communicate()` per turn because it closes stdin. [3]

Use subprocess argument arrays rather than constructing a shell command from team-supplied text. Track each container's ID/name. On a deadline or fault, explicitly remove the container, then reap the Docker CLI process; killing the CLI alone is not the arena's cleanup strategy.

7. Container profile

Assume a native linux/amd64 tournament host for the example. Confirm the actual architecture before the workshop; if the host is ARM, agree on linux/arm64 and update every manifest/build command. Use the same native platform for all entrants; avoid turning CPU emulation into part of the competition.

Proposed per-bot profile: one CPU quota, 256 MiB RAM with no additional swap, 64 processes, read-only root filesystem, 16 MiB temporary /tmp, non-root UID/GID 10001, all Linux capabilities dropped, no privilege escalation, no host mounts, no published ports, and no network beyond isolated loopback. No secrets, shared volumes, Docker socket, host namespaces, or arbitrary team-selected runtime flags. Bots may use any language that works within this profile. Preflight the limits on the actual host; Docker notes some constraints depend on host kernel support. [1,4]

Example launch, using a unique name for every bot/match and an already-local image:

```
docker run --rm -i --init --pull=never \  
  --name vtarena-m01-a \  
  --network=none \  
  --read-only \  
  --tmpfs /tmp:rw,noexec,nosuid,size=16m \  
  --cpus=1 \  
  --memory=256m --memory-swap=256m \  
  --pids-limit=64 \  
  --user=10001:10001 \  
  --cap-drop=ALL \  
  --security-opt=no-new-privileges=true \  
  vt-bots/pattern-hawk:1.0
```

The arena implements readiness/move timeouts and byte limits; these are not implied by the Docker flags. Use the resolved immutable local image ID instead of a mutable tag in scored runs. Keep default seccomp/AppArmor protections where available. [1,5]

These controls are intended for a trusted classroom, not a hostile public code-execution service. Containers are not a complete isolation guarantee; Docker documents configuration and kernel risks. Use a disposable host/VM without sensitive data, inspect submissions before loading/building, and never give bot containers Docker access. Public adversarial submissions would require a stronger, separately engineered isolation and admission design. [5]

8. Submission and registration

Each team supplies `bot.py` (or its equivalent source), `Dockerfile`, `bot.json`, a short README, and either a prebuilt image archive or a preapproved build context. Source sharing makes this a classroom software exercise rather than an opaque untrusted-image competition.

Example manifest:

```
{
  "id": "pattern-hawk",
  "display_name": "Pattern Hawk",
  "team": "Team A",
  "image": "vt-bots/pattern-hawk:1.0",
  "protocol": "vt-arena/1",
  "game": "rps-v1",
  "platform": "linux/amd64"
}
```

IDs contain lowercase letters, digits, and hyphens, up to 32 characters. Names/team labels are plain text. Manifests cannot supply mounts, environment variables, entrypoint overrides, network settings, or arbitrary Docker options. Validate uniqueness and the declared protocol/game/platform. Resolve and record the local image ID, then freeze all registered image IDs before scored play.

Example build and transfer, after adapting the tag/name and confirming the host platform:

```
docker build --platform linux/amd64 -t vt-bots/pattern-hawk:1.0 .
docker image save -o pattern-hawk.tar vt-bots/pattern-hawk:1.0
# Transfer the archive and bot.json to the designated organizer.
docker image load -i pattern-hawk.tar
```

Save/load transfers images without needing a new registry. Use `save -o` and `load -i` rather than binary shell redirection. Docker documents these image archive operations. [6,7]

Proposed commands for the arena team to implement, **not existing commands in this kit**:

```
python arena.py verify --manifest bot.json
python arena.py match --a bots/a.json --b bots/b.json --rounds 100
python arena.py tournament --manifest-dir bots --legs 2 --rounds 100
python arena.py replay --events runs/tournament.jsonl
```

9. Qualification and acceptance tests

Qualification is separate from scoring. A bot must launch under the actual profile, respond to start, complete 100 sequential legal turns, use the correct IDs/rounds, remain within byte/time budgets, emit no stdout diagnostics, terminate on end/EOF, and repeat the same moves on the same request transcript and seed. Use coherent history derived from committed moves, not invented feedback. A self-reported deterministic flag is not sufficient.

The arena's tests must cover all nine move pairs; swapped/mirrored score views; no release of the current move before both commits; an independently assessed deadline for each bot; malformed/extra/oversized/unflushed output; a hung bot; early exit; single and double forfeits; seed isolation; image freezing; six pairings/four matches per bot; scoring/tie rules; cancellation without bot penalty; and container cleanup on success, failure, and Ctrl+C. A log replay must reproduce standings without rerunning containers.

Required log data: protocol/game version, tournament/match/round identifiers, immutable image IDs, organizer-private seeds, committed moves and outcomes, round scores, response elapsed times, fault types, match awards, and final standings. Do not log or display a current move before the commitment barrier. A replay of committed events is authoritative; reexecuting a bot is a diagnostic experiment that may be affected by timing.

10. Workshop run sheet

Time	Arena team	Bot teams
0–10 min	Confirm host, contract, fixtures, responsibilities	Run reference bot and inspect protocol
10–30 min	Launch two reference containers; implement one scored round	Implement strategy and local protocol tests
30–50 min	Complete a 100-round match, limits, logging, cleanup	Build image and pass qualification
50–65 min	Add six-match scheduling and standings	Practice, inspect failures, revise
65–75 min	Integrate and freeze image IDs	Submit final image/manifest and a one-sentence strategy hypothesis
75–90 min	Run tournament, show selected replay rounds, discuss	Observe, explain predictions, compare failures

For 60 minutes, use one round robin (three matches) as an explicitly announced shortened schedule, a terminal scoreboard, and no browser replay. Finalize that configuration before scored play. For 90 minutes, use the six-match schedule above. Full resource-profile qualification is never traded away for animation.

Pre-session preparation should include a working Docker host, local base image, reference bot, these fixtures/schemas, and a simple smoke-test command. A blank-sheet arena with lifecycle control, robust pipe handling, a polished UI, and new strategies is too much to make all of those mandatory in one workshop. The reliable match runner is the acceptance target; animation is a stretch goal.

11. Team prompts

Arena team

Build the arena described in SPEC.md. You own only the referee, Docker lifecycle, protocol validator, tournament scheduler, event log, and presentation. Use two persistent no-network bot containers per match and vt-arena/1 JSON lines over standard streams. First make the reference bot complete a qualified match; then implement fault tests, six-match scheduling, and standings. Resolve moves only after both commits. Never accept team-supplied runtime flags or mount the Docker socket into a bot. Use the fixed resources, bounded concurrent stream readers, and explicit container cleanup. Keep the protocol unchanged. The browser view is optional; a working terminal scoreboard and replayable log are required. Report tests run and anything not verified on the target Docker host.

Each bot team

Build a containerized bot for SPEC.md without changing the vt-arena/1 protocol. Start from bot.py and replace choose_move(history, rng). Use only completed-round history and your own supplied seed; no live API calls, network access, persistent cross-match state, or external randomness. Print only one correctly identified JSON response per request and flush it. Write diagnostics only to stderr. Supply source, tests, Dockerfile, and bot.json. State a strategy hypothesis, test it against predictable and random practice opponents, and submit an image that passes the organizer's qualification under the exact runtime profile.

12. Official implementation references

All accessed September 16, 2026. These references support implementation mechanisms, not the workshop-specific game rules or feasibility estimates.

1. Docker: docker container run. <https://docs.docker.com/reference/cli/docker/container/run/>
2. Docker: None network driver. <https://docs.docker.com/engine/network/drivers/none/>
3. Python: asyncio subprocesses. <https://docs.python.org/3/library/asyncio-subprocess.html>
4. Docker: Resource constraints. https://docs.docker.com/engine/containers/resource_constraints/
5. Docker: Docker Engine security. <https://docs.docker.com/engine/security/>
6. Docker: docker image save. <https://docs.docker.com/reference/cli/docker/image/save/>
7. Docker: docker image load. <https://docs.docker.com/reference/cli/docker/image/load/>
8. Docker: Dockerfile reference. <https://docs.docker.com/reference/dockerfile/>
9. Docker Official Image: Python. https://hub.docker.com/_/python